

GRC Compass: A Retrieval-Augmented Cybersecurity Compliance Assistant

Written By:

*Hamza Habib
Kamar El-hayouni
Saad Foda*

Supervised By:

*Prof. Calin Anton
Prof. Mohamed El-Hajj*

Date:

April 20, 2026

1. Introduction and Context

Organizations are required to follow cybersecurity frameworks such as the NIST Cybersecurity Framework and PCI DSS, as well as provincial regulations such as the Health Information Act. These documents are often long, technical, and difficult to navigate efficiently.

Users, such as analysts and auditors, need to locate specific controls or requirements quickly. In practice, this process is slow and requires manually searching through hundreds of pages. This increases the risk of missing important information and reduces productivity.

This project addresses that problem by building a chatbot that allows users to ask questions directly about compliance documents. Instead of reading entire files, users can query the system in natural language and receive relevant answers based on the uploaded material.

2. Problem Statement

Compliance documents are large and unstructured. A single framework can exceed 100 pages. Organizations often rely on multiple frameworks simultaneously.

This creates three main issues:

- Information retrieval is slow
- Relevant sections are difficult to locate
- Users may misinterpret or overlook requirements

For example, finding all references to incident response across multiple documents can take several minutes per query. This becomes inefficient when repeated across many tasks.

3. Motivation

The goal of this project is to reduce the time and effort required to work with compliance documents.

A system that can quickly retrieve relevant information improves efficiency and reduces manual effort. It also helps ensure responses are based on actual documentation rather than assumptions.

This is especially important in cybersecurity, where accuracy and traceability matter.

4. Objectives

The project was designed with the following objectives:

- Build a chatbot that answers questions using uploaded documents
- Ensure responses are based only on document content
- Support PDF uploads and manual text input
- Use vector search to retrieve relevant information
- Provide fast and consistent responses
- Improve usability through a simple web interface

5. Methodology and System Design

5.1 System Overview

The system follows a Retrieval-Augmented Generation (RAG) approach, combining a retrieval component with a generative model to produce more accurate, grounded responses (Lewis et al., 2020). RAG retrieves relevant documents and uses them as context during generation, improving factual accuracy and reducing hallucinations.

Flow:

User uploads a document

- Text is extracted
 - Text is split into chunks
 - Each chunk is converted into embeddings
 - Stored in database
 - User asks a question
 - Query is embedded
 - Similar chunks are retrieved
 - Model generates an answer using the retrieved context
-

5.2 RAG Architecture

The chatbot is designed to answer only using the uploaded documents. This aligns with RAG models, which retrieve relevant documents and use them as context during response generation (Lewis et al., 2020).

When a query is submitted:

1. The system converts the query into a vector

2. It compares this vector with stored embeddings
3. It retrieves the most similar chunks
4. These chunks are passed to the model
5. The model generates a response based only on this context

If no relevant information is found, the system returns:

“I don’t know based on the uploaded documents.”

This prevents the model from generating unsupported answers.

5.3 Text Processing

Documents are processed as follows:

- PDFs are converted to raw text
- Text is normalized
- Text is split into chunks of about 800 characters

Chunking ensures that:

- The model does not exceed token limits
 - Retrieval is more precise
 - Large documents can be handled efficiently
-

5.4 Embeddings and Similarity Search

Each text chunk is converted into a vector using an embedding model.

- Model used: Gemini embedding model
- Vector size: 3072 dimensions

To find relevant content, the system uses cosine similarity.

Cosine similarity measures how close two vectors are in direction. A value closer to 1 indicates higher similarity.

The system applies:

- Similarity threshold: 0.55
- Top-k retrieval: 8 chunks

Only chunks above the threshold are considered. The top 8 results are passed to the model.

6. Data

6.1 Tools & Technologies

Data

- PDF compliance documents — NIST CSF, PCI DSS, ISO 27001, and Canadian cybersecurity frameworks

- Text extracted via pdf-parse and chunked into ~800-character segments for precise retrieval
- Stored as 3072-dimensional vector embeddings in PostgreSQL

Algorithms

- Text chunking (sliding window) · Vector embeddings · Cosine similarity search · Top-k retrieval

Tools & Libraries

- Next.js (frontend + API routes) · Drizzle ORM · PostgreSQL + pg vector
- Google Gemini (2.5 Flash-Lite for generation, Gemini embedding model for vectors)

7. Implementation and Development Lifecycle

7.1 Chatbot Initialization and Infrastructure

The initial stage of the project focused on establishing a strong backend connection. We used Drizzle ORM to interface with our database, ensuring a type-safe, efficient communication layer between the application and the vector store. Having this foundation is critical for handling the complex data schemas required by a RAG system.

7.2 Global State Management and Persistence

To prevent React state from unmounting during navigation, we implemented a Global Context Provider using the React Context API. By “lifting” the chatbot state to the root layout level, the messages array remains persistent regardless of which dashboard module (e.g., NIST CSF 2.0 Coverage) the user is currently viewing. Additionally, we implemented SessionStorage Hydration. This ensures that the conversation history is synchronized with the browser’s temporary storage, allowing the audit trail to persist even if the user refreshes their browser window, a critical requirement for maintaining continuity during long compliance reviews.

8. Implementation and Development Log

8.1 LLM Benchmarking and the Pivot to Cloud

Early development involved hosting models locally via Ollama to prioritize data privacy. During this stage, we performed extensive hallucination testing to determine how different models handled out-of-bounds query questions not answered in the provided PDF.

The Challenge: Local execution experienced significant latency, often taking 30–60 seconds to generate a response due to high resource demands on the local machine.

The Solution: To optimize the user experience and increase the inference speed, we transitioned to the **Gemini 2.5 Flash Lite** model. This transition, facilitated by the **Vercel AI SDK**, resulted in near-instantaneous response rates and superior long-context window management.

8.2 Refining Conversational Logic and Natural Language

A common issue with strict RAG systems is their tendency to be overly dismissive of standard social cues. Initially, our bot would respond to simple greetings like “Hi” or “Hello” with an “I don't know” error because those words were not in the uploaded cybersecurity PDFs. To resolve this, we implemented a **preprocessing layer** in the system prompt. This allowed the bot to recognize and respond naturally to small talk while maintaining its strict technical compliance grounding, creating a more professional interface.

9. Regulatory Research and Domain Knowledge

9.1 NIST Framework Analysis and Knowledge

To ensure the chatbot provided accurate audits, we conducted an in-depth analysis of the **NIST Cybersecurity Framework**. This involved manually analyzing and documenting the framework requirements to determine the best way to “chunk” the data for the AI pipeline. Our research ensured the bot understood the specific distinct of NIST controls, allowing it to provide grounded answers rather than generic summaries.

10. Results

The system was evaluated based on response speed, retrieval accuracy, and overall usability.

One of the most significant improvements was in **response time**. During early development, locally hosted models (via Ollama) produced responses in approximately 30–60 seconds per query. After transitioning to the Gemini 2.5 Flash Lite model, response times became near-instant, significantly improving the user experience.

In terms of **retrieval accuracy**, the use of vector embeddings and cosine similarity enabled the system to return relevant sections from uploaded documents consistently. The top-k retrieval approach ensured that the most relevant chunks were passed to the model, reducing the likelihood of irrelevant or misleading responses.

The system also demonstrated strong performance in handling **natural language queries**. Users could ask questions in plain language, such as requesting summaries of compliance requirements or identifying relevant controls. The chatbot successfully interpreted these queries and returned context-aware responses grounded in the uploaded material.

Additionally, implementing a fallback response (“I don’t know based on the uploaded documents”) ensured the system did not generate unsupported answers. This improved the chatbot's reliability, particularly in a compliance context where accuracy is critical.

From a usability perspective, integrating a web-based interface with PDF upload functionality enabled users to interact with the system in a straightforward, efficient manner. The ability to upload and query documents in real time reduced the need for manual searching and improved workflow efficiency.

Overall, the system achieved its primary objective of reducing the time required to locate relevant compliance information while maintaining accurate, document-based responses.

10.1 Technical Challenges and Solutions

Our first major technical hurdle was the scale of the regulatory documents. Frameworks, specifically those sourced from the Canadian Centre for Cyber Security, often exceed 500 pages. Standard AI context windows often crash or lose critical detail when processing files this large.

We resolved this by implementing recursive text chunking and vector embeddings. By breaking the text into 800-character segments, we ensured the system could perform a cosine similarity search to retrieve only the most relevant sections. This architecture allows the bot to provide precise citations back to specific sections of the source document without overwhelming the model's memory.

Second, we addressed a critical cybersecurity concern: Data Privacy. While cloud models like Gemini 2.5 Flash Lite offer high speed and superior context management, many firms require “air-gapped” solutions to keep sensitive compliance data off the public internet.

To solve this, we engineered an architecture that is fully compatible with **Ollama** for local hosting. By maintaining a modular backend, we ensured the system could be deployed on a private internal server. This meets strict data residency and privacy requirements for organizations that cannot utilize third-party cloud APIs for sensitive regulatory audits.

10.2 Type Safety and SDK Integration

A recurring challenge involved ensuring strict **TypeScript** compliance between our internal data structures and the Gemini model's multimodal “Parts” format. The Vercel AI SDK requires specific message objects that differ from our initial database schema. We resolved this by engineering a

custom transformation layer that maps retrieved database chunks to the SDK-compliant format, ensuring the UI remains resilient and avoiding runtime **TypeErrors** during the RAG pipeline execution.

11. Conclusion

11.1 Limitations

While the system improves efficiency in navigating compliance documents, it has several limitations.

First, the chatbot is restricted to the content of uploaded documents. If relevant information is missing from the dataset, the system cannot provide an answer and will return a fallback response. This ensures accuracy but limits completeness.

Second, the effectiveness of retrieval depends on how the documents are processed. The chunking approach (approximately 800 characters per chunk) may split related information across different sections, thereby reducing the quality of retrieved results.

Third, the system relies on cosine similarity with a fixed threshold (0.55). While this helps filter irrelevant content, it may also exclude useful information if the similarity score falls slightly below the threshold.

Additionally, the chatbot does not perform advanced reasoning across multiple documents. It retrieves the most relevant chunks but may not fully synthesize complex relationships between different frameworks or regulations.

Finally, system performance is dependent on the quality and structure of the uploaded documents. Poorly formatted or incomplete files can negatively impact both retrieval accuracy and response quality.

11.2 Future Work

While the current system provides a functional and efficient compliance assistant, several areas for improvement can enhance both usability and performance.

Another important enhancement is the inclusion of **detailed source citations** in responses. Although the system retrieves relevant document chunks, it does not currently display the exact

source or section to the user. Providing clear references to the original documents would increase transparency and trust, allowing users to verify the information more easily.

The system can also be expanded to support **additional file formats** beyond PDFs and plain text. For example, allowing uploads of Word documents or structured data files would make the platform more flexible and accessible for different types of users and organizations.

In terms of performance, the **document processing pipeline can be improved**. The current chunking method uses fixed-size text segments, which may not always preserve meaningful context. Future work could involve implementing more advanced chunking techniques, such as splitting by paragraph or semantic boundaries, to improve retrieval accuracy.

Additionally, the system could be enhanced with more advanced retrieval and reasoning capabilities. This includes improving how the chatbot synthesizes information across multiple documents and frameworks, enabling more comprehensive, context-aware responses.

Overall, these improvements aim to make the system more scalable, user-friendly, and effective in supporting real-world cybersecurity compliance tasks.

12. References

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in Neural Information Processing Systems, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>